

Selective Re-Execution of Invalidated Subgraphs

The piece most supervisor demos skip. Rerunning the whole graph on every edit is where the token bill comes from.

The Problem

Most LangGraph demos show a linear pipeline -- user says "change X", the whole graph reruns from scratch. That's fine for a tutorial but brutal in production where each agent call burns an LLM round-trip with tools. Five agents x multiple tool calls each x 3-4 user revisions per session = a compounding token bill with no added value.

How AI Travel Genie Solves It

The system uses **cascade-based selective invalidation** -- a split-responsibility design where the LLM and deterministic rules each handle what they're good at:

Split Responsibility

Responsibility	Handled By	Why
What changed in the user's request	LLM (_parse_modification())	Natural language understanding -- "make it cheaper" needs i
Which agents to re-run	Deterministic cascade rules (CASCADE_R	Business logic -- predictable, testable, no hallucination risk

Cascade Rules

When a `TravelRequest` field changes, only the agents whose outputs depend on that field are invalidated:

```

CASCADE_RULES = {
  "destination":      [all 5 agents],
  "origin":           [flight, hotel, budget],
  "start_date" / "end_date": [flight, hotel, itinerary, budget],
  "budget_usd":       [hotel, budget],
  "travelers":        [flight, hotel, budget],
  "preferences":      [hotel, itinerary, budget],
  "special_requirements": [itinerary, budget],
}

```

The Mechanism

- User modifies the plan** -> `human_review` sets `plan_status: "revising"` and routes back to `intent_parser`
- `parse_intent` detects modification` -- checks `plan_status` and finds an existing `travel_request`
- LLM merges the change** -- `_parse_modification()` calls the LLM to produce an updated `TravelRequest`, with a keyword-based fallback if the LLM fails
- Diff detection** -- `_diff_requests()` compares old vs. new request field-by-field
- Cascade computation** -- `_compute_cascade()` maps changed fields -> invalidated agent output fields
- Selective nulling** -- only invalidated fields are set to `None`:

```

for field in invalidated:
    result[field] = None

```

Fields **not** in the invalidated list retain their existing values.

- Router naturally skips unaffected agents** -- `supervisor_router` checks each field for `None` sequentially. Agents whose outputs survived the cascade are skipped because their fields are still populated.

Concrete Example

User says: "Increase the budget to \$6,000"

Step	Result
<code>_diff_requests()</code>	<code>{"budget_usd"}</code>
<code>_compute_cascade({"budget_usd"})</code>	<code>["hotel_options", "budget_summary"]</code>
Fields nulled	<code>hotel_options = None, budget_summary = None</code>
Fields preserved	<code>destination_research, flight_options, itinerary</code>
Agents re-run	Hotel Search, Budget/Currency (2 of 5)

Agents skipped

Destination, Flight, Activity (3 of 5)

Result: 60% fewer LLM calls, tool executions, and tracing spans on a single edit.

User says: "Change the destination to Bali"

Step	Result
_diff_requests()	{"destination"}
_compute_cascade({"destination"})	All 5 agent fields
Agents re-run	All 5 (destination change affects everything)

The cascade correctly identifies that a destination change requires a full re-plan -- no under-invalidation.

Why This Design Matters

Token Cost Savings

Across a multi-turn conversation with 3-4 revisions, the savings compound significantly:

Scenario	Naïve (full re-run)	Selective	Savings
"Increase budget"	5 agent calls	2 agent calls	60%
"Add a traveler"	5 agent calls	3 agent calls	40%
"Change dates"	5 agent calls	4 agent calls	20%
"Change destination"	5 agent calls	5 agent calls	0% (correct)
Typical 3-edit session	15 agent calls	~9 agent calls	~40%

Each "agent call" involves an LLM reasoning loop (ReAct sub-graph) with 2-3 tool invocations -- so the actual LLM token savings are multiplicative.

No Special Re-Run Mode

The same `supervisor_router` handles both first-run and selective re-execution:

```
def supervisor_router(state):
    if state.get("destination_research") is None:
        return "destination_research"
    if state.get("flight_options") is None:
        return "flight_search"
    # ... checks each field ...
    return "aggregator"
```

There's one code path, one place where ordering is defined, one thing to test. The router doesn't know or care whether it's a first run or a modification -- it just routes to the next agent whose output is missing.

Predictable and Testable

Because cascade rules are deterministic (not LLM-decided), they're unit-testable:

```
def test_budget_change_cascade():
    result = _compute_cascade({"budget_usd"})
    assert result == ["hotel_options", "budget_summary"]
    assert "destination_research" not in result
    assert "flight_options" not in result
```

No flaky tests, no prompt sensitivity, no "the LLM decided to skip the hotel agent this time."

Key Takeaway

This pattern -- **cascade rules + null-based routing** -- separates a demo from something you'd actually run with real users and real API bills. The LLM does what it's good at (understanding natural language modifications), and deterministic code handles the consequences (which agents to invalidate). That split is what makes multi-turn agent systems economically viable.